

第二讲：语言与文法

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 3 月 10 日



福州大学
FUZHOU UNIVERSITY

- ① 语言
- ② 文法
- ③ 总结与作业



目录

① 语言

语言是什么？

程序设计语言的诞生过程

形式化语言

程序设计语言

② 文法

③ 总结与作业



目录

① 语言

语言是什么？

程序设计语言的诞生过程

形式化语言

程序设计语言

② 文法

③ 总结与作业



语言示例

- ## ● 自然语言



语言示例

- 自然语言
 - 我看见了他拿着望远镜。



语言示例

- 自然语言
 - 我看见了他拿着望远镜。
 - Python 是一种蛇，也是一种编程语言。



语言示例

- 自然语言
 - 我看见了他拿着望远镜。
 - Python 是一种蛇，也是一种编程语言。
- 程序设计语言



语言示例

- 自然语言
 - 我看见了他拿着望远镜。
 - Python 是一种蛇，也是一种编程语言。
- 程序设计语言
 - `print('Hello World')`



语言示例

- 自然语言
 - 我看见了他拿着望远镜。
 - Python 是一种蛇，也是一种编程语言。
- 程序设计语言
 - `print('Hello World')`
 - `cout << "Hello World" << endl;`



语言的定义



语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。



语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法



语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法

用途 表达思想、传递情感



语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法

用途 表达思想、传递情感

特点 能产性、任意性和分层结构



语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法

用途 表达思想、传递情感

特点 能产性、任意性和分层结构

- **能产性 (Productivity)**：有限的词汇和规则可以组合出无限多的句子。

语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法

用途 表达思想、传递情感

特点 能产性、任意性和分层结构

- 能产性 (Productivity)：有限的词汇和规则可以组合出无限多的句子。
- 任意性 (Arbitrariness)：符号（词）与所指事物之间没有必然联系。

语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法

用途 表达思想、传递情感

特点 能产性、任意性和分层结构

- 能产性 (Productivity)：有限的词汇和规则可以组合出无限多的句子。
- 任意性 (Arbitrariness)：符号（词）与所指事物之间没有必然联系。
- **分层性 (Hierarchy)**：声音（音素）组成词，词组成句子。

语言的定义

语言是一种**符号系统**，是人类的交际和思维**工具**。

构成 语音、词汇和语法

用途 表达思想、传递情感

特点 能产性、任意性和分层结构

- 能产性 (Productivity)：有限的词汇和规则可以组合出无限多的句子。
- 任意性 (Arbitrariness)：符号（词）与所指事物之间没有必然联系。
- 分层性 (Hierarchy)：声音（音素）组成词，词组成句子。

形式 口语、书面语

目录

① 语言

语言是什么？

程序设计语言的诞生过程

形式化语言

程序设计语言

② 文法

③ 总结与作业



人类

图 1: 程序设计语言的诞生过程

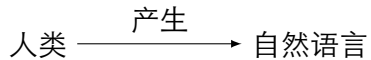


图 1: 程序设计语言的诞生过程

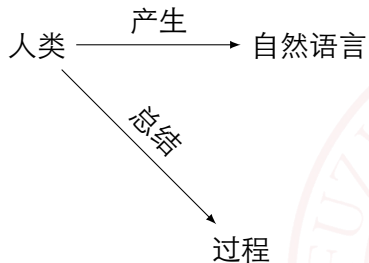


图 1: 程序设计语言的诞生过程

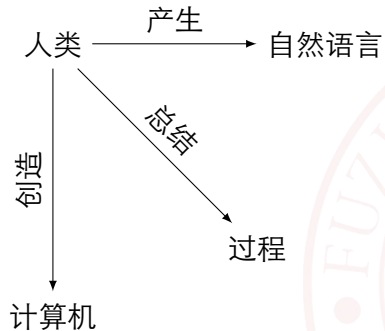


图 1: 程序设计语言的诞生过程

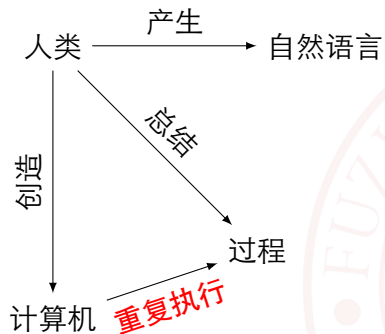


图 1: 程序设计语言的诞生过程

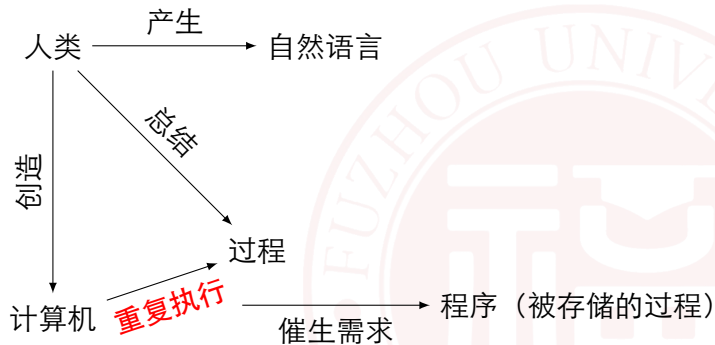


图 1: 程序设计语言的诞生过程

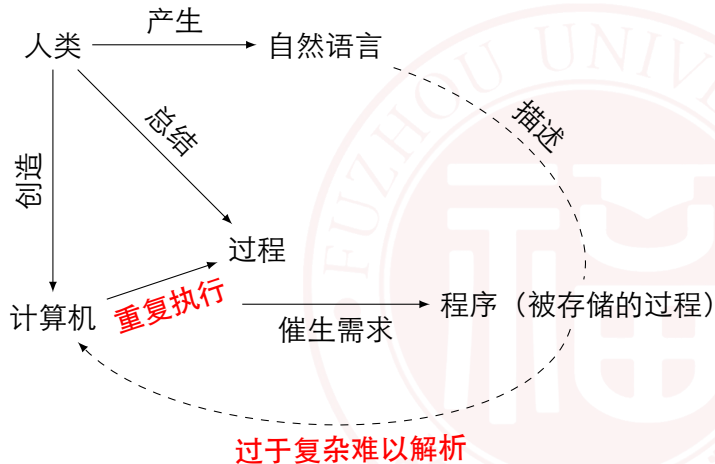


图 1: 程序设计语言的诞生过程

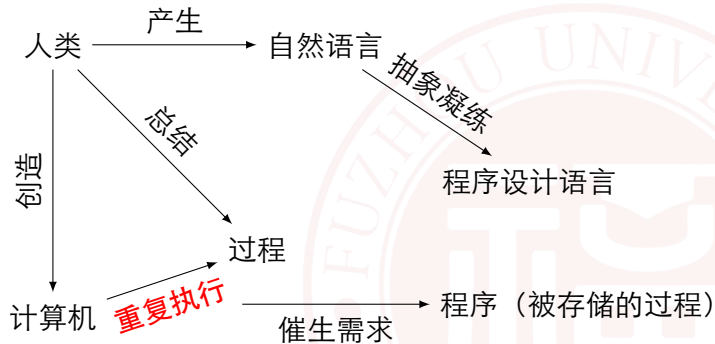


图 1: 程序设计语言的诞生过程

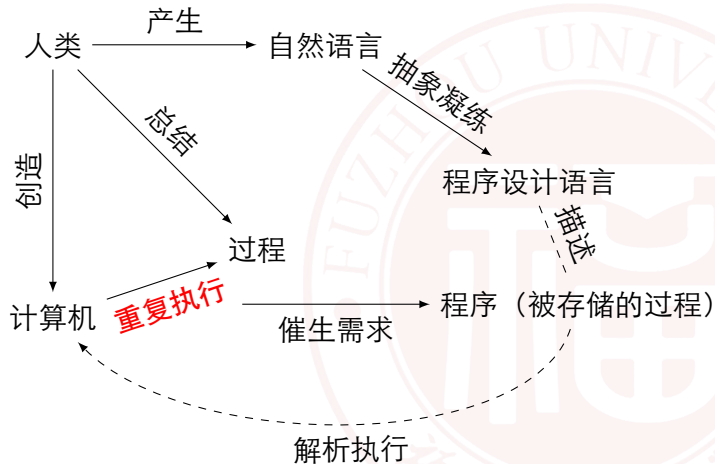
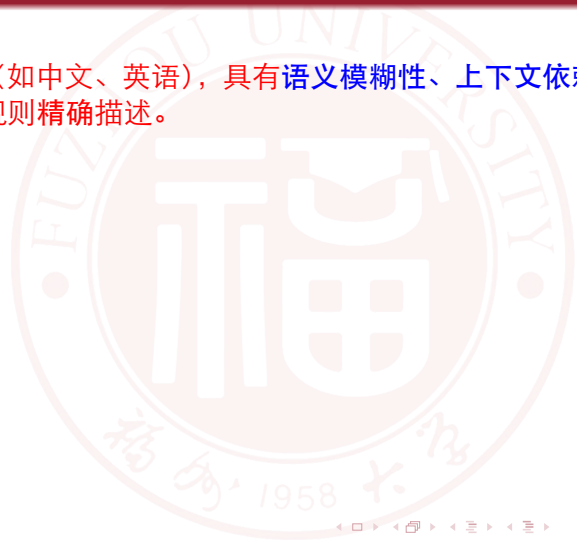


图 1: 程序设计语言的诞生过程

语言的特征

- 人类用于交流的口语和书面语（如中文、英语），具有语义模糊性、上下文依赖性、文化约定性，无法用有限规则精确描述。



语言的特征

- 人类用于交流的口语和书面语（如中文、英语），具有**语义模糊性**、**上下文依赖性**、**文化约定性**，**无法**用**有限**规则**精确**描述。

表 1: 自然语言与形式化语言的对比

特征	自然语言	形式化语言
----	------	-------

语言的特征

- 人类用于交流的口语和书面语（如中文、英语），具有**语义模糊性**、**上下文依赖性**、**文化约定性**，**无法**用**有限**规则**精确**描述。

表 1: 自然语言与形式化语言的对比

特征	自然语言	形式化语言
精确性	模糊（例：“苹果”指水果或公司）	无歧义（例：C 语言 <code>int x</code> ; 仅表示整型变量）

语言的特征

- 人类用于交流的口语和书面语（如中文、英语），具有**语义模糊性**、**上下文依赖性**、**文化约定性**，**无法**用**有限**规则**精确**描述。

表 1: 自然语言与形式化语言的对比

特征	自然语言	形式化语言
精确性	模糊（例：“苹果”指水果或公司）	无歧义（例：C 语言 <code>int x</code> ; 仅表示整型变量）
规则	无固定规则（依赖语境）	严格数学规则（文法定义）

语言的特征

- 人类用于交流的口语和书面语（如中文、英语），具有**语义模糊性**、**上下文依赖性**、**文化约定性**，**无法用有限规则精确**描述。

表 1: 自然语言与形式化语言的对比

特征	自然语言	形式化语言
精确性	模糊（例：“苹果”指水果或公司）	无歧义（例：C 语言 <code>int x</code> ; 仅表示整型变量）
规则	无固定规则（依赖语境）	严格数学规则（文法定义）
可计算性	无法被机器直接解析	可被编译器精确解析

语言的特征

- 人类用于交流的口语和书面语（如中文、英语），具有**语义模糊性**、**上下文依赖性**、**文化约定性**，**无法用有限规则精确**描述。

表 1: 自然语言与形式化语言的对比

特征	自然语言	形式化语言
精确性	模糊（例：“苹果”指水果或公司）	无歧义（例：C 语言 <code>int x</code> ; 仅表示整型变量）
规则	无固定规则（依赖语境）	严格数学规则（文法定义）
可计算性	无法被机器直接解析	可被编译器精确解析

- 形式化语言是通过**字母表** (Σ) 和**文法** (**Grammar**, G) 严格定义的字符串集合，用于计算机科学，具有**固定规则**、**无歧义性**、**可计算性**。

目录

① 语言

语言是什么？

程序设计语言的诞生过程

形式化语言

程序设计语言

② 文法

③ 总结与作业



语言的形式化定义

- 在形式语言理论中，语言是字母表（**Alphabet**）上字符串的集合。



语言的形式化定义

- 在形式语言理论中，语言是**字母表（Alphabet）**上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，……}



语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，……}
 - 英文的字符集是**26 个字母**，……



语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，.....}
 - 英文的字符集是**26 个字母**，.....
 -

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，.....}
 - 英文的字符集是**26 个字母**，.....
 -
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ϵ ）。

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，.....}
 - 英文的字符集是 **26 个字母**，.....
 -
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ϵ ）。
 - $\Sigma = \{a, b\}$

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，……}
 - 英文的字符集是**26 个字母**，……
 - ……
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ε ）。
 - $\Sigma = \{a, b\} \rightarrow \Sigma^* = \{\varepsilon, a, b, aa, ab, b, bb, \dots\}$

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，.....}
 - 英文的字符集是**26 个字母**，.....
 -
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ε ）。
 - $\Sigma = \{a, b\} \rightarrow \Sigma^* = \{\varepsilon, a, b, aa, ab, b, bb, \dots\}$
 - 语言 $L_1 = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, \dots\}$

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，……}
 - 英文的字符集是**26 个字母**，……
 - ……
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ε ）。
 - $\Sigma = \{a, b\} \rightarrow \Sigma^* = \{\varepsilon, a, b, aa, ab, b, bb, \dots\}$
 - 语言 $L_1 = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, \dots\}$
 - 语言 $L_2 = \{a^n \mid n \geq 0\} = \{\varepsilon, a, aa, aaa, \dots\}$

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，.....}
 - 英文的字符集是**26 个字母**，.....
 -
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ε ）。
 - $\Sigma = \{a, b\} \rightarrow \Sigma^* = \{\varepsilon, a, b, aa, ab, b, bb, \dots\}$
 - 语言 $L_1 = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, \dots\}$
 - 语言 $L_2 = \{a^n \mid n \geq 0\} = \{\varepsilon, a, aa, aaa, \dots\}$
- 思考题：若 Σ^4 表示长度为 4 的字符串的集合，那么 Σ^∞ 表示什么？它是可列集吗？请试着证明一下。

语言的形式化定义

- 在形式语言理论中，语言是**字母表 (Alphabet)** 上字符串的集合。
 - 中文的字符集是**汉字**，形成的集合有{你，我，他，吃了没，……}
 - 英文的字符集是**26 个字母**，……
 - ……
- 数学化表达：若 Σ 是有限的符号集合（字母表），则语言 $L \subseteq \Sigma^*$ ，其中 Σ^* 表示 Σ 上所有字符串的集合（包括空串 ε ）。
 - $\Sigma = \{a, b\} \rightarrow \Sigma^* = \{\varepsilon, a, b, aa, ab, b, bb, \dots\}$
 - 语言 $L_1 = \{a^n b^n \mid n \geq 1\} = \{ab, aabb, aaabbb, \dots\}$
 - 语言 $L_2 = \{a^n \mid n \geq 0\} = \{\varepsilon, a, aa, aaa, \dots\}$
- 思考题：若 Σ^4 表示长度为 4 的字符串的集合，那么 Σ^∞ 表示什么？它是可列集吗？请试着证明一下。
- 思考题：如果 Σ 是一个可列集合，那么 Σ^* 是可列集吗？请试着证明一下。

语言的运算

- 并: $L_1 \cup L_2 = \{s \mid s \in L_1 \vee s \in L_2\}$



语言的运算

- 并: $L_1 \cup L_2 = \{s \mid s \in L_1 \vee s \in L_2\}$
- 交: $L_1 \cap L_2 = \{s \mid s \in L_1 \wedge s \in L_2\}$



语言的运算

- 并: $L_1 \cup L_2 = \{s \mid s \in L_1 \vee s \in L_2\}$
- 交: $L_1 \cap L_2 = \{s \mid s \in L_1 \wedge s \in L_2\}$
- 接: $L_1 || L_2 = \{s_1 s_2 \mid s_1 \in L_1 \wedge s_2 \in L_2\}$

语言的运算

- 并: $L_1 \cup L_2 = \{s \mid s \in L_1 \vee s \in L_2\}$
- 交: $L_1 \cap L_2 = \{s \mid s \in L_1 \wedge s \in L_2\}$
- 接: $L_1 || L_2 = \{s_1 s_2 \mid s_1 \in L_1 \wedge s_2 \in L_2\}$
- 幂: $L^n = L^{n-1} || L$, 特别地 $L^0 = \{\epsilon\}$ (包含空串的集合)

语言的运算

- 并: $L_1 \cup L_2 = \{s \mid s \in L_1 \vee s \in L_2\}$
- 交: $L_1 \cap L_2 = \{s \mid s \in L_1 \wedge s \in L_2\}$
- 接: $L_1 || L_2 = \{s_1 s_2 \mid s_1 \in L_1 \wedge s_2 \in L_2\}$
- 幂: $L^n = L^{n-1} || L$, 特别地 $L^0 = \{\varepsilon\}$ (包含空串的集合)
- 闭包: $L^* = \bigcup_{i=1}^{\infty} L^i$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

① $L_1 \cup L_2$



运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

① $L_1 \cup L_2$

答案:

① $\{a, ab, b, ba\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)

答案:

- ① $\{a, ab, b, ba\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)

答案:

- ① $\{a, ab, b, ba\}$
- ② $\{ab, aba, abb, abba\}$

运算示例

已知：

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算：

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)
- ③ L_1^2 (平方/幂)

答案:

- 1 $\{a, ab, b, ba\}$
- 2 $\{ab, aba, abb, abba\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)
- ③ L_1^2 (平方/幂)

答案:

- ① $\{a, ab, b, ba\}$
- ② $\{ab, aba, abb, abba\}$
- ③ $\{aa, aab, aba, abab\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)
- ③ L_1^2 (平方/幂)
- ④ L_1^* (写出闭包的前几项)

答案:

- ① $\{a, ab, b, ba\}$
- ② $\{ab, aba, abb, abba\}$
- ③ $\{aa, aab, aba, abab\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)
- ③ L_1^2 (平方/幂)
- ④ L_1^* (写出闭包的前几项)

答案:

- ① $\{a, ab, b, ba\}$
- ② $\{ab, aba, abb, abba\}$
- ③ $\{aa, aab, aba, abab\}$
- ④ $\{\epsilon, a, ab, aa, aab, aba, abab, \dots\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)
- ③ L_1^2 (平方/幂)
- ④ L_1^* (写出闭包的前几项)

小提示

- 连接不满足交换律: $L_1 || L_2 \neq L_2 || L_1$ (连接的顺序会影响结果)。

答案:

- ① $\{a, ab, b, ba\}$
- ② $\{ab, aba, abb, abba\}$
- ③ $\{aa, aab, aba, abab\}$
- ④ $\{\epsilon, a, ab, aa, aab, aba, abab, \dots\}$

运算示例

已知:

- 字母表 $\Sigma = \{a, b\}$
- 语言 $L_1 = \{a, ab\}$
- 语言 $L_2 = \{b, ba\}$

请计算:

- ① $L_1 \cup L_2$
- ② $L_1 || L_2$ (连接)
- ③ L_1^2 (平方/幂)
- ④ L_1^* (写出闭包的前几项)

小提示

- 连接不满足交换律: $L_1 || L_2 \neq L_2 || L_1$ (连接的顺序会影响结果)。
- 闭包必含空串: 无论 L 是什么, L^* 永远包含 ε 。

答案:

- ① $\{a, ab, b, ba\}$
- ② $\{ab, aba, abb, abba\}$
- ③ $\{aa, aab, aba, abab\}$
- ④ $\{\varepsilon, a, ab, aa, aab, aba, abab, \dots\}$

目录

① 语言

语言是什么？

程序设计语言的诞生过程

形式化语言

程序设计语言

② 文法

③ 总结与作业



程序设计语言的三要素

- 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”



程序设计语言的三要素

- 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”

- 语义

- 程序的含义
- 代码的执行逻辑
- 表达式的计算结果
- 决定了程序“做什么”

程序设计语言的三要素

• 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”

• 语义

- 程序的含义
- 代码的执行逻辑
- 表达式的计算结果
- 决定了程序“做什么”

• 标准库

- 语言提供的基础功能
- 常用的数据结构和算法
- 系统接口和工具函数
- 决定了开发效率

程序设计语言的三要素

• 语法

- 程序的结构形式
- 代码的书写规则
- 符号的组合方式
- 决定了程序是否“合法”
- “语法糖”——让代码更易读但不改变语义的语法特性

• 语义

- 程序的含义
- 代码的执行逻辑
- 表达式的计算结果
- 决定了程序“做什么”

• 标准库

- 语言提供的基础功能
- 常用的数据结构和算法
- 系统接口和工具函数
- 决定了开发效率

目录

① 语言

② 文法

语言与文法

文法的层次

语法树与推导

文法的二义性

③ 总结与作业



目录

① 语言

② 文法

语言与文法

文法的层次

语法树与推导

文法的二义性

③ 总结与作业



如何描述一个语言?

- 枚举：按照字典序，枚举语言集合中的每一个元素



集合中的每一个元素

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选

如何描述一个语言?

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$



如何描述一个语言?

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$

如何描述一个语言?

- 枚举：按照字典序，枚举语言集合中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$
- 生成：按照规则开始生成字符串

如何描述一个语言?

- 枚举：按照字典序，枚举语言集合中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$
- 生成：按照规则开始生成字符串
 - ① 定义初始串 ab

如何描述一个语言?

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$
- 生成：按照规则开始生成字符串
 - ① 定义初始串 ab
 - ② 定义替换规则 $ab \rightarrow aabb$

如何描述一个语言?

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$
- 生成：按照规则开始生成字符串
 - ① 定义初始串 ab
 - ② 定义替换规则 $ab \rightarrow aabb$
 - ③ 对初始串应用替换规则得到 $aabb$

如何描述一个语言?

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$
- 生成：按照规则开始生成字符串
 - ① 定义初始串 ab
 - ② 定义替换规则 $ab \rightarrow aabb$
 - ③ 对初始串应用替换规则得到 $aabb$
 - ④ 继续对生成的串执行规则得到 $aaabbb$

合中的每一个元素

$s \in \Sigma^*$

abb

$aaabbb$

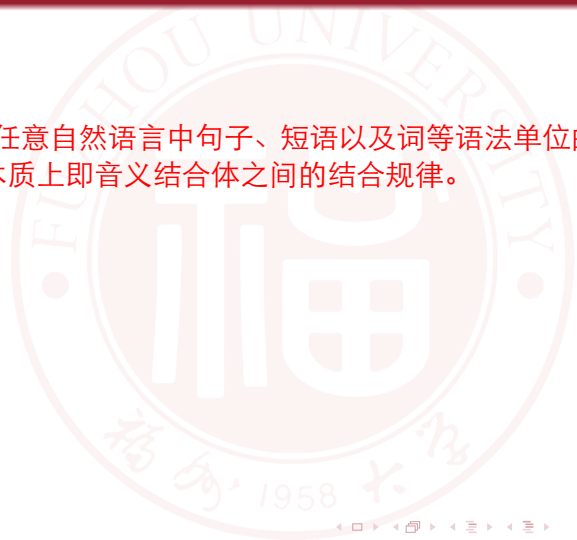
- 1 定义初始串 ab
- 2 定义替换规则 $ab \rightarrow aabb$
- 3 对初始串应用替换规则得到 $aabb$
- 4 继续对生成的串执行规则得到 $aaabbb$
- 5

如何描述一个语言?

- 枚举：按照字典序，枚举语言集中的每一个元素
- 过滤：按规则在全集上筛选
 - 正偶数： $\{2i \mid i \in \mathbb{N}\}$
 - 以 a 开始的所有字符串： $\{as \mid s \in \Sigma^*\}$
- 生成：按照规则开始生成字符串
 - ① 定义初始串 ab
 - ② 定义替换规则 $ab \rightarrow aabb$
 - ③ 对初始串应用替换规则得到 $aabb$
 - ④ 继续对生成的串执行规则得到 $aaabbb$
 - ⑤
 - ⑥ 在应用任意次规则后均可以停止，并把得到的串加入语言集合中

文法

- 语法（grammar，也称文法）是任意自然语言中句子、短语以及词等语法单位的语法结构与语法意义的规律，本质上即音义结合体之间的结合规律。



文法

- 语法（grammar，也称文法）是任意自然语言中句子、短语以及词等语法单位的语法结构与语法意义的规律，本质上即音义结合体之间的结合规律。
- 形式文法：在形式语言理论中，文法（formal grammar）是形式语言中字符串的一套产生式规则（production rule）。

文法

- 语法（grammar，也称文法）是任意自然语言中句子、短语以及词等语法单位的语法结构与语法意义的规律，本质上即音义结合体之间的结合规律。
- 形式文法：在形式语言理论中，文法（formal grammar）是形式语言中字符串的一套产生式规则（production rule）。
- 形式文法描述了如何用语言的字母表生成符合句法（syntax）的有效的字符串。

文法

- 语法（grammar，也称文法）是任意自然语言中句子、短语以及词等语法单位的语法结构与语法意义的规律，本质上即音义结合体之间的结合规律。
- 形式文法：在形式语言理论中，文法（formal grammar）是形式语言中字符串的一套产生式规则（production rule）。
- 形式文法描述了如何用语言的字母表生成符合句法（syntax）的有效的字符串。
- 文法不描述字符串的含义，也不描述在任何上下文中可以用它们做什么——只描述它们的形式。

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

- V 是非终结符的集合

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

- V 是非终结符的集合
- Σ 是终结符集合，即字符集

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

- V 是非终结符的集合
- Σ 是终结符集合，即字符集
- P 是产生式的集合

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

- V 是非终结符的集合
- Σ 是终结符集合，即字符集
- P 是产生式的集合
- 产生式是形如 $S \rightarrow abc$ 式子，表示可以将一个串的中左侧部分替换为右侧部分

形式文法

定义

一个文法可以用一个四元组描述： $G = (V, \Sigma, P, S)$ ，其中

- V 是非终结符的集合
- Σ 是终结符集合，即字符集
- P 是产生式的集合
- 产生式是形如 $S \rightarrow abc$ 式子，表示可以将一个串的中左侧部分替换为右侧部分
- S 是起始符号（Initial Symbol，亦称初始符号），决定产生式的替换由哪个符号开始

形式文法示例

例

文法 $G = (V = \{S\}, \Sigma = \{a, b\}, P = \{S \rightarrow aSb \mid \varepsilon\}, S)$ ，尝试**推导 (derive)** 字符串 $aabb$

形式文法示例

例

文法 $G = (V = \{S\}, \Sigma = \{a, b\}, P = \{S \rightarrow aSb \mid \varepsilon\}, S)$ ，尝试**推导 (derive)** 字符串 $aabb$

- 应用规则 $S \rightarrow aSb$, $S \Rightarrow aSb$

形式文法示例

例

文法 $G = (V = \{S\}, \Sigma = \{a, b\}, P = \{S \rightarrow aSb \mid \varepsilon\}, S)$ ，尝试**推导 (derive)** 字符串 $aabb$

- 应用规则 $S \rightarrow aSb$, $S \Rightarrow aSb$
- 应用规则 $S \rightarrow aSb$, $aSb \Rightarrow aaSbb$

形式文法示例

例

文法 $G = (V = \{S\}, \Sigma = \{a, b\}, P = \{S \rightarrow aSb \mid \varepsilon\}, S)$ ，尝试**推导 (derive)** 字符串 $aabb$

- 应用规则 $S \rightarrow aSb$, $S \Rightarrow aSb$
- 应用规则 $S \rightarrow aSb$, $aSb \Rightarrow aaSbb$
- 应用规则 $S \rightarrow \varepsilon$, $aaSbb \Rightarrow aabb$

文法 G 能生成的语言是什么？

形式文法示例

例

文法 $G = (V = \{S\}, \Sigma = \{a, b\}, P = \{S \rightarrow aSb \mid \varepsilon\}, S)$, 尝试**推导 (derive)** 字符串 $aabb$

- 应用规则 $S \rightarrow aSb$, $S \Rightarrow aSb$
- 应用规则 $S \rightarrow aSb$, $aSb \Rightarrow aaSbb$
- 应用规则 $S \rightarrow \varepsilon$, $aaSbb \Rightarrow aabb$

文法 G 能生成的语言是什么? $\{a^n b^n \mid n \geq 0\}$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$
$$NP \rightarrow Det \ N \mid Det \ N \ PP$$
$$VP \rightarrow V \ NP \mid V$$
$$PP \rightarrow P \ NP$$
$$Det \rightarrow a \mid an \mid the$$
$$N \rightarrow cat \mid dog \mid house$$
$$V \rightarrow saw \mid chased$$
$$P \rightarrow in \mid on$$


自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$
$$NP \rightarrow Det \ N \mid Det \ N \ PP$$
$$VP \rightarrow V \ NP \mid V$$
$$PP \rightarrow P \ NP$$
$$Det \rightarrow a \mid an \mid the$$
$$N \rightarrow cat \mid dog \mid house$$
$$V \rightarrow saw \mid chased$$
$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$
- $\Rightarrow the \ cat \ saw \ NP$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$
- $\Rightarrow the \ cat \ saw \ NP$
- $\Rightarrow the \ cat \ saw \ Det \ N$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$
- $\Rightarrow the \ cat \ saw \ NP$
- $\Rightarrow the \ cat \ saw \ Det \ N$
- $\Rightarrow the \ cat \ saw \ the \ N$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$
- $\Rightarrow the \ cat \ saw \ NP$
- $\Rightarrow the \ cat \ saw \ Det \ N$
- $\Rightarrow the \ cat \ saw \ the \ N$
- $\Rightarrow the \ cat \ saw \ the \ dog$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$$S \rightarrow NP \ VP$$

$$NP \rightarrow Det \ N \mid Det \ N \ PP$$

$$VP \rightarrow V \ NP \mid V$$

$$PP \rightarrow P \ NP$$

$$Det \rightarrow a \mid an \mid the$$

$$N \rightarrow cat \mid dog \mid house$$

$$V \rightarrow saw \mid chased$$

$$P \rightarrow in \mid on$$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$
- $\Rightarrow the \ cat \ saw \ NP$
- $\Rightarrow the \ cat \ saw \ Det \ N$
- $\Rightarrow the \ cat \ saw \ the \ N$
- $\Rightarrow the \ cat \ saw \ the \ dog$

自然语言文法示例：生成英文句子

例

文法 $G = (V, \Sigma, P, S)$ 生成简单英文句子：

$S \rightarrow NP \ VP$

$NP \rightarrow Det \ N \mid Det \ N \ PP$

$VP \rightarrow V \ NP \mid V$

$PP \rightarrow P \ NP$

$Det \rightarrow a \mid an \mid the$

$N \rightarrow cat \mid dog \mid house$

$V \rightarrow saw \mid chased$

$P \rightarrow in \mid on$

生成句子 “The cat saw the dog”：

- $S \Rightarrow NP \ VP$
- $\Rightarrow Det \ N \ VP$
- $\Rightarrow the \ N \ VP$
- $\Rightarrow the \ cat \ VP$
- $\Rightarrow the \ cat \ V \ NP$
- $\Rightarrow the \ cat \ saw \ NP$
- $\Rightarrow the \ cat \ saw \ Det \ N$
- $\Rightarrow the \ cat \ saw \ the \ N$
- $\Rightarrow the \ cat \ saw \ the \ dog$

这个文法还能生成 “a dog chased the house”，但 **无法生成 “the dog saw cat”**（缺少冠词），体现文法的规则约束性。

C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$



C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$
- $declaration \rightarrow (funcdecl|vardecl)^*$



C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$
- $declaration \rightarrow (funcdecl|vardecl)^*$
- $definition \rightarrow (funcdef|vardef|typedef)^*$

C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$
- $declaration \rightarrow (funcdecl|vardecl)^*$
- $definition \rightarrow (funcdef|vardef|typedef)^*$
- $vardef \rightarrow typename\ identifier = initvalue ;$

C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$
- $declaration \rightarrow (funcdecl|vardecl)^*$
- $definition \rightarrow (funcdef|vardef|typedef)^*$
- $vardef \rightarrow typename\ identifier = initvalue ;$
- $typename \rightarrow int|char|float|double|void|identifier$

C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$
- $declaration \rightarrow (funcdecl|vardecl)^*$
- $definition \rightarrow (funcdef|vardef|typedef)^*$
- $vardef \rightarrow typename\ identifier = initvalue ;$
- $typename \rightarrow int|char|float|double|void|identifier$
- $initvalue \rightarrow number|char|string$

C 语言的形式文法（部分）

- $program \rightarrow (declaration|definition)^*$
- $declaration \rightarrow (funcdecl|vardecl)^*$
- $definition \rightarrow (funcdef|vardef|typedef)^*$
- $vardef \rightarrow typename\ identifier = initvalue ;$
- $typename \rightarrow int|char|float|double|void|identifier$
- $initvalue \rightarrow number|char|string$
- $funcdef \rightarrow typename\ identifier (parameterlist)\ block$

如何书写一个文法?

- 箭头写法



如何书写一个文法?

- 箭头写法
- 巴科斯范式 (Backus Normal Form, BNF), 又称为巴科斯-诺尔范式 (Backus-Naur Form, BNF)
 - $\langle \text{src1} \rangle \langle \text{src2} \rangle \langle \text{src3} \rangle \dots ::= \langle \text{dst1} \rangle \langle \text{dst2} \rangle \langle \text{dst3} \rangle \dots$

如何书写一个文法?

- 箭头写法
- 巴科斯范式 (Backus Normal Form, BNF), 又称为巴科斯-诺尔范式 (Backus-Naur Form, BNF)
 - $\langle \text{src1} \rangle \langle \text{src2} \rangle \langle \text{src3} \rangle \dots ::= \langle \text{dst1} \rangle \langle \text{dst2} \rangle \langle \text{dst3} \rangle \dots$
- 扩展巴科斯范式 (Extended Backus-Naur Form, EBNF)
 - $\langle \text{src1} \rangle \langle \text{src2} \rangle \langle \text{src3} \rangle \dots ::= \langle \text{dst1} \rangle \langle \text{dst2} \rangle \langle \text{dst3} \rangle \dots$
 - 允许使用正则表达式的语法糖, 如*、+、?等

例题：从文法推导语言集合

题目：给定文法 G ，产生式如下：

$$S \rightarrow aA \quad A \rightarrow bA \mid b$$

求 $L(G)$ 的集合表达式。

例题：从文法推导语言集合

题目：给定文法 G ，产生式如下：

$$S \rightarrow aA \quad A \rightarrow bA \mid b$$

求 $L(G)$ 的集合表达式。

分析过程：

- 分析 S ：
 - 必须以 a 开头。
 - 后续部分由 A 决定。

例题：从文法推导语言集合

题目：给定文法 G ，产生式如下：

$$S \rightarrow aA \quad A \rightarrow bA \mid b$$

求 $L(G)$ 的集合表达式。

分析过程：

- 分析 S ：
 - 必须以 a 开头。
 - 后续部分由 A 决定。
- 分析 A ：
 - $A \rightarrow b$ (生成 1 个 b)
 - $A \rightarrow bA \rightarrow bb$ (生成 2 个 b)
 - $A \rightarrow bA \rightarrow bbb$ (生成 3 个 b)
 - **规律**： A 生成 $\{b^n \mid n \geq 1\}$

例题：从文法推导语言集合

题目：给定文法 G ，产生式如下：

$$S \rightarrow aA \quad A \rightarrow bA \mid b$$

求 $L(G)$ 的集合表达式。

分析过程：

- 分析 S ：
 - 必须以 a 开头。
 - 后续部分由 A 决定。
- 分析 A ：
 - $A \rightarrow b$ (生成 1 个 b)
 - $A \rightarrow bA \rightarrow bb$ (生成 2 个 b)
 - $A \rightarrow bA \rightarrow bbb$ (生成 3 个 b)
 - **规律**： A 生成 $\{b^n \mid n \geq 1\}$

结论：

$$L(G) = \{ab^n \mid n \geq 1\}$$

例题：从文法推导语言集合

题目：给定文法 G ，产生式如下：

$$S \rightarrow aA \quad A \rightarrow bA \mid b$$

求 $L(G)$ 的集合表达式。

分析过程：

- 分析 S ：
 - 必须以 a 开头。
 - 后续部分由 A 决定。
- 分析 A ：
 - $A \rightarrow b$ (生成 1 个 b)
 - $A \rightarrow bA \rightarrow bb$ (生成 2 个 b)
 - $A \rightarrow bA \rightarrow bbb$ (生成 3 个 b)
 - **规律**： A 生成 $\{b^n \mid n \geq 1\}$

结论：

$$L(G) = \{ab^n \mid n \geq 1\}$$

- 合法句子： $ab, abb, abbb$

例题：从文法推导语言集合

题目：给定文法 G ，产生式如下：

$$S \rightarrow aA \quad A \rightarrow bA \mid b$$

求 $L(G)$ 的集合表达式。

分析过程：

- 分析 S ：
 - 必须以 a 开头。
 - 后续部分由 A 决定。
- 分析 A ：
 - $A \rightarrow b$ (生成 1 个 b)
 - $A \rightarrow bA \rightarrow bb$ (生成 2 个 b)
 - $A \rightarrow bA \rightarrow bbb$ (生成 3 个 b)
 - 规律**： A 生成 $\{b^n \mid n \geq 1\}$

结论：

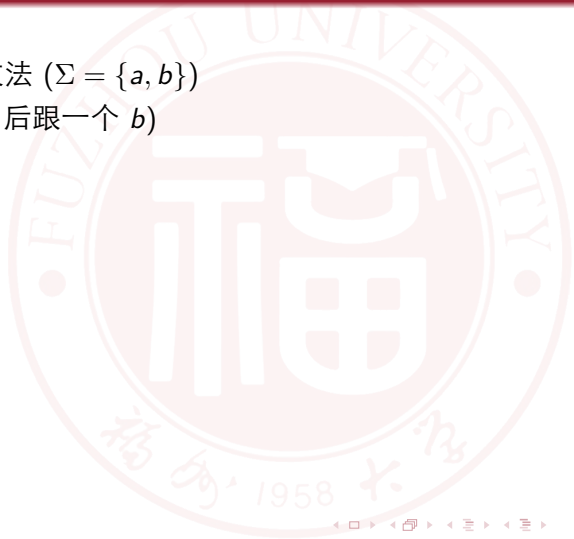
$$L(G) = \{ab^n \mid n \geq 1\}$$

- 合法句子： $ab, abb, abbb$
- 非法句子： a (少 b), b (少 a), aab (多 a)

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)



例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)
- ③ $L_3 = \{a^{2k} \mid k \geq 0\} \cup \{b^{2k+1} \mid k \geq 0\}$ (偶数个 a 或 奇数个 b)

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)
- ③ $L_3 = \{a^{2k} \mid k \geq 0\} \cup \{b^{2k+1} \mid k \geq 0\}$ (偶数个 a 或 奇数个 b)

解题技巧

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)
- ③ $L_3 = \{a^{2k} \mid k \geq 0\} \cup \{b^{2k+1} \mid k \geq 0\}$ (偶数个 a 或 奇数个 b)

解题技巧

- 无限重复 ($n \geq 0$): 使用递归 $S \rightarrow xS \mid \varepsilon$

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)
- ③ $L_3 = \{a^{2k} \mid k \geq 0\} \cup \{b^{2k+1} \mid k \geq 0\}$ (偶数个 a 或 奇数个 b)

解题技巧

- 无限重复 ($n \geq 0$): 使用递归 $S \rightarrow xS \mid \varepsilon$
- 至少一个 ($n \geq 1$): 使用递归 $S \rightarrow xS \mid x$

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)
- ③ $L_3 = \{a^{2k} \mid k \geq 0\} \cup \{b^{2k+1} \mid k \geq 0\}$ (偶数个 a 或 奇数个 b)

解题技巧

- 无限重复 ($n \geq 0$): 使用递归 $S \rightarrow xS \mid \varepsilon$
- 至少一个 ($n \geq 1$): 使用递归 $S \rightarrow xS \mid x$
- 成对出现 (偶数): 每次生成两个 $S \rightarrow xxS \mid \varepsilon$

例题：从语言集合构造文法

任务：为以下语言构造上下文无关文法 ($\Sigma = \{a, b\}$)

- ① $L_1 = \{a^n b \mid n \geq 0\}$ (任意个 a 后跟一个 b)
- ② $L_2 = \{(ab)^n \mid n \geq 0\}$ (ab 重复, 含空串)
- ③ $L_3 = \{a^{2k} \mid k \geq 0\} \cup \{b^{2k+1} \mid k \geq 0\}$ (偶数个 a 或 奇数个 b)

解题技巧

- 无限重复 ($n \geq 0$): 使用递归 $S \rightarrow xS \mid \varepsilon$
- 至少一个 ($n \geq 1$): 使用递归 $S \rightarrow xS \mid x$
- 成对出现 (偶数): 每次生成两个 $S \rightarrow xxS \mid \varepsilon$
- 并集: $S \rightarrow A \mid B$

挑战题

构造一个 01 字符串文法，使其生成的语言不包含任何有“100”子串的字符串。

目录

① 语言

② 文法

语言与文法

文法的层次

语法树与推导

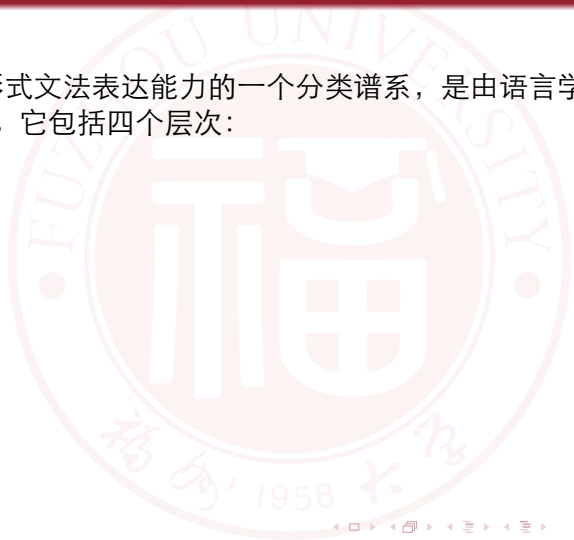
文法的二义性

③ 总结与作业



文法的层次

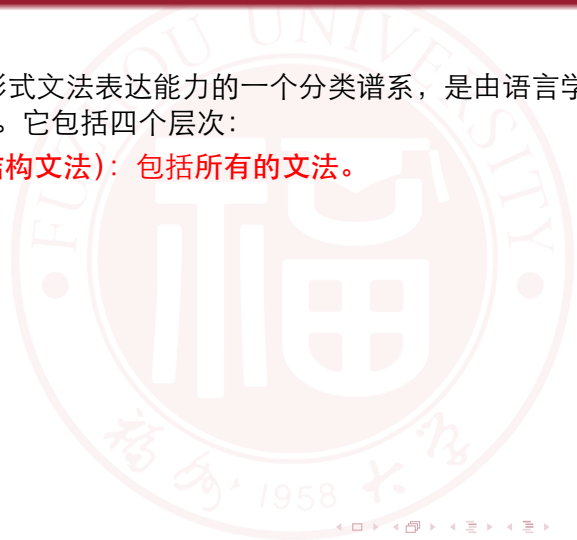
乔姆斯基体系是计算机科学中刻画形式文法表达能力的一个分类谱系，是由语言学家诺姆·乔姆斯基于 1956 年提出的。它包括四个层次：



文法的层次

乔姆斯基体系是计算机科学中刻画形式文法表达能力的一个分类谱系，是由语言学家诺姆·乔姆斯基于 1956 年提出的。它包括四个层次：

- **0-型文法（无限制文法或短语结构文法）：** 包括所有的文法。



文法的层次

乔姆斯基体系是计算机科学中刻画形式文法表达能力的一个分类谱系，是由语言学家诺姆·乔姆斯基于 1956 年提出的。它包括四个层次：

- **0-型文法（无限制文法或短语结构文法）**：包括所有的文法。
- **1-型文法（上下文相关文法）**：生成上下文相关语言。这种文法的产生式规则取如 $\alpha A \beta \rightarrow \alpha \gamma \beta$ 一样的形式。

文法的层次

乔姆斯基体系是计算机科学中刻画形式文法表达能力的一个分类谱系，是由语言学家诺姆·乔姆斯基于 1956 年提出的。它包括四个层次：

- **0-型文法（无限制文法或短语结构文法）**：包括所有的文法。
- **1-型文法（上下文相关文法）**：生成上下文相关语言。这种文法的产生式规则取如 $\alpha A \beta \rightarrow \alpha \gamma \beta$ 一样的形式。
- **2-型文法（上下文无关文法）**：生成上下文无关语言。这种文法的产生式规则取如 $A \rightarrow \gamma$ 一样的形式。

文法的层次

乔姆斯基体系是计算机科学中刻画形式文法表达能力的一个分类谱系，是由语言学家诺姆·乔姆斯基于 1956 年提出的。它包括四个层次：

- **0-型文法（无限制文法或短语结构文法）**：包括所有的文法。
- **1-型文法（上下文相关文法）**：生成上下文相关语言。这种文法的产生式规则取如 $\alpha A \beta \rightarrow \alpha \gamma \beta$ 一样的形式。
- **2-型文法（上下文无关文法）**：生成上下文无关语言。这种文法的产生式规则取如 $A \rightarrow \gamma$ 一样的形式。
- **3-型文法（正则文法）**：生成正则语言。产生式的左侧只能包含一个非终结符号，产生式的右侧只能是空串、一个终结符号或者一个终结符号后随一个非终结符号。

不同层次文法与对应语言示例

① 正则文法可生成 $\{a^m b^n \mid m, n \geq 0\}$



不同层次文法与对应语言示例

- ① 正则文法可生成 $\{a^m b^n \mid m, n \geq 0\}$
- ② 上下文无关文法可生成 $\{a^n b^n \mid n \geq 0\}$

不同层次文法与对应语言示例

- ① 正则文法可生成 $\{a^m b^n \mid m, n \geq 0\}$
- ② 上下文无关文法可生成 $\{a^n b^n \mid n \geq 0\}$
- ③ 上下文相关文法可生成 $\{a^n b^n c^n \mid n \geq 0\}$

不同层次文法与对应语言示例

- ① 正则文法可生成 $\{a^m b^n \mid m, n \geq 0\}$
- ② 上下文无关文法可生成 $\{a^n b^n \mid n \geq 0\}$
- ③ 上下文相关文法可生成 $\{a^n b^n c^n \mid n \geq 0\}$
- ④ 无限制文法可生成 $\{a^n b^n c^n d^n \mid n \geq 0\}$

文法的对比

文法层次	文法名称	处理机模型	定义的语言
3-型文法	正则文法 (Regular Grammar)	有限状态自动机 (Finite Automata)	正则语言 (Regular Language)
2-型文法	上下文无关文法 (Context-Free Grammar)	下推自动机 (Pushdown Automata)	上下文无关语言 (Context-Free Language)
1-型文法	上下文相关文法 (Context-Sensitive Grammar)	线性有界自动机 (Linear bounded Automata)	上下文相关语言 (Context-Sensitive Language)
0-型文法	无限制文法 (Unrestricted Grammar)	图灵机 (Turning Machine)	递归可枚举语言 (Recursively Enumerable Languages)

目录

① 语言

② 文法

语言与文法

文法的层次

语法树与推导

文法的二义性

③ 总结与作业



文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。



文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。



文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>

文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>
 ⇒ 小明<谓语><宾语>

文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>
 ⇒ 小明<谓语><宾语>
 ⇒ 小明<动词><宾语>

文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>
 ⇒ 小明<谓语><宾语>
 ⇒ 小明<动词><宾语>
 ⇒ 小明学习<宾语>

文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>
 ⇒ 小明<谓语><宾语>
 ⇒ 小明<动词><宾语>
 ⇒ 小明学习<宾语>
 ⇒ 小明学习数学

- **句子 (Sentence)**: 由文法生成的字符串, 即从起始符号经过若干次推导得到的终结符号串。

文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>
 ⇒ 小明<谓语><宾语>
 ⇒ 小明<动词><宾语>
 ⇒ 小明学习<宾语>
 ⇒ 小明学习数学

- **句子 (Sentence)**: 由文法生成的字符串, 即从起始符号经过若干次推导得到的终结符号串。
- **句型 (Sentential Form)**: 在推导过程中出现的包含非终结符号和终结符号的字符串。

文法相关重要定义

- **推导 (Derivation)**: 根据文法的产生式规则, 从起始符号出发, 通过一系列替换操作生成字符串的过程。

<句子> ⇒ <主语><谓语><宾语>
 ⇒ 小明<谓语><宾语>
 ⇒ 小明<动词><宾语>
 ⇒ 小明学习<宾语>
 ⇒ 小明学习数学

- **句子 (Sentence)**: 由文法生成的字符串, 即从起始符号经过若干次推导得到的终结符号串。
- **句型 (Sentential Form)**: 在推导过程中出现的包含非终结符号和终结符号的字符串。
- **语法树 (Parsing Tree)**: 表示推导过程的树形结构, 根节点为起始符号, 叶节点为终结符号, 内部节点为非终结符号。

语法树示例

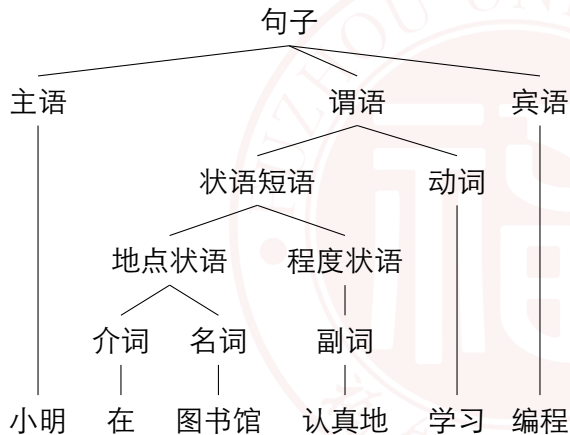


图 2: 一个中文句子的语法树示例

推导、直接推导、归约的定义

<句子> $\Rightarrow$ <主语><谓语><宾语>
 $\Rightarrow$ 小明<谓语><宾语>
 $\Rightarrow$ 小明<动词><宾语>
 $\Rightarrow$ 小明学习<宾语>
 $\Rightarrow$ 小明学习数学

- 设文法 $G = (V_T, V_N, P, S)$,

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$,

推导、直接推导、归约的定义

<句子> $\Rightarrow$ <主语><谓语><宾语>
 $\Rightarrow$ 小明<谓语><宾语>
 $\Rightarrow$ 小明<动词><宾语>
 $\Rightarrow$ 小明学习<宾语>
 $\Rightarrow$ 小明学习数学

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$,

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$,

推导、直接推导、归约的定义

<句子> $\Rightarrow$ <主语><谓语><宾语>
 $\Rightarrow$ 小明<谓语><宾语>
 $\Rightarrow$ 小明<动词><宾语>
 $\Rightarrow$ 小明学习<宾语>
 $\Rightarrow$ 小明学习数学

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**,

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。
- 若存在一系列式子 $\alpha_1, \alpha_2, \dots, \alpha_n$,

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。
- 若存在一系列式子 $\alpha_1, \alpha_2, \dots, \alpha_n$, 使得 $\alpha_1 = \alpha$, $\alpha_n = \beta$, 且对每个 i ($1 \leq i < n$), 都有 $\alpha_i \Rightarrow \alpha_{i+1}$,

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。
- 若存在一系列式子 $\alpha_1, \alpha_2, \dots, \alpha_n$, 使得 $\alpha_1 = \alpha$, $\alpha_n = \beta$, 且对每个 i ($1 \leq i < n$), 都有 $\alpha_i \Rightarrow \alpha_{i+1}$, 则从 α 到 β 的过程称为**推导**,

推导、直接推导、归约的定义

$\langle \text{句子} \rangle \Rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{谓语} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明} \langle \text{动词} \rangle \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习} \langle \text{宾语} \rangle$
 $\Rightarrow \text{小明学习数学}$

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。
- 若存在一系列式子 $\alpha_1, \alpha_2, \dots, \alpha_n$, 使得 $\alpha_1 = \alpha$, $\alpha_n = \beta$, 且对每个 i ($1 \leq i < n$), 都有 $\alpha_i \Rightarrow \alpha_{i+1}$, 则从 α 到 β 的过程称为**推导**, 记为 $\alpha \xRightarrow{*} \beta$ 。

推导、直接推导、归约的定义

<句子> $\Rightarrow$ <主语><谓语><宾语>
 $\Rightarrow$ 小明<谓语><宾语>
 $\Rightarrow$ 小明<动词><宾语>
 $\Rightarrow$ 小明学习<宾语>
 $\Rightarrow$ 小明学习数学

- 设文法 $G = (V_T, V_N, P, S)$, 若 $\alpha A \beta \in V^*$, $A \in V_N$, 且 $A \rightarrow \gamma \in P$, 则称从 $\alpha A \beta$ 到 $\alpha \gamma \beta$ 的一步替换为**直接推导**, 记为 $\alpha A \beta \Rightarrow \alpha \gamma \beta$ 。
- 若存在一系列式子 $\alpha_1, \alpha_2, \dots, \alpha_n$, 使得 $\alpha_1 = \alpha$, $\alpha_n = \beta$, 且对每个 i ($1 \leq i < n$), 都有 $\alpha_i \Rightarrow \alpha_{i+1}$, 则从 α 到 β 的过程称为**推导**, 记为 $\alpha \xRightarrow{*} \beta$ 。
- 归约 (reduction): 推导的逆过程。

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则
 - $E \Rightarrow E + T$

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则
 - $E \Rightarrow E + T$
 - $T \Rightarrow T * F$

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则

- $E \Rightarrow E + T$

- $T \Rightarrow T * F$

- 推导 ($\stackrel{*}{\Rightarrow}$): 应用多次规则序列

$$E \Rightarrow E + T \Rightarrow T + T$$

$$\Rightarrow F + T \Rightarrow \text{id} + T$$

$$\Rightarrow \text{id} + T * F \Rightarrow \text{id} + \text{id} * F$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则

- $E \Rightarrow E + T$

- $T \Rightarrow T * F$

- 推导 ($\Rightarrow^*$): 应用多次规则序列

$$E \Rightarrow E + T \Rightarrow T + T$$

$$\Rightarrow F + T \Rightarrow \text{id} + T$$

$$\Rightarrow \text{id} + T * F \Rightarrow \text{id} + \text{id} * F$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$

- 记作: $E \Rightarrow^* \text{id} + \text{id} * \text{id}$

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

2. 归约 (Reduction) (从 句子 到 S)

- 直接推导 ($\Rightarrow$): 仅应用一次规则

- $E \Rightarrow E + T$

- $T \Rightarrow T * F$

- 推导 ($\Rightarrow^*$): 应用多次规则序列

$$E \Rightarrow E + T \Rightarrow T + T$$

$$\Rightarrow F + T \Rightarrow \text{id} + T$$

$$\Rightarrow \text{id} + T * F \Rightarrow \text{id} + \text{id} * F$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$

- 记作: $E \Rightarrow^* \text{id} + \text{id} * \text{id}$

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则

- $E \Rightarrow E + T$

- $T \Rightarrow T * F$

- 推导 ($\Rightarrow^*$): 应用多次规则序列

$$E \Rightarrow E + T \Rightarrow T + T$$

$$\Rightarrow F + T \Rightarrow \text{id} + T$$

$$\Rightarrow \text{id} + T * F \Rightarrow \text{id} + \text{id} * F$$

$$\Rightarrow \text{id} + \text{id} * \text{id}$$

- 记作: $E \Rightarrow^* \text{id} + \text{id} * \text{id}$

2. 归约 (Reduction) (从 句子 到 S)

$$\text{id} + \text{id} * \text{id} \Leftarrow \text{id} + \text{id} * F \Leftarrow \text{id} + F * F$$

$$\Leftarrow \text{id} + T * F \Leftarrow \text{id} + T$$

$$\Leftarrow \text{id} + T$$

$$\Leftarrow F + T \Leftarrow T + T$$

$$\Leftarrow E + T \Leftarrow E$$

从推导到归约

文法 $G[E]$:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow \text{id}$$

1. 推导 (Derivation) (从 S 到 句子)

- 直接推导 ($\Rightarrow$): 仅应用一次规则

- $E \Rightarrow E + T$
- $T \Rightarrow T * F$

- 推导 ($\stackrel{*}{\Rightarrow}$): 应用多次规则序列

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow T + T \\ &\Rightarrow F + T \Rightarrow \text{id} + T \\ &\Rightarrow \text{id} + T * F \Rightarrow \text{id} + \text{id} * F \\ &\Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

- 记作: $E \stackrel{*}{\Rightarrow} \text{id} + \text{id} * \text{id}$

2. 归约 (Reduction) (从 句子 到 S)

$$\begin{aligned} \text{id} + \text{id} * \text{id} &\Leftarrow \text{id} + \text{id} * F \Leftarrow \text{id} + F * F \\ &\Leftarrow \text{id} + T * F \Leftarrow \text{id} + T \\ &\Leftarrow \text{id} + T \\ &\Leftarrow F + T \Leftarrow T + T \\ &\Leftarrow E + T \Leftarrow E \end{aligned}$$

记作: $\text{id} + \text{id} * \text{id} \stackrel{*}{\Leftarrow} E$

目录

① 语言

② 文法

语言与文法

文法的层次

语法树与推导

文法的二义性

③ 总结与作业



例

设有文法 $G[E]$:

$$E \rightarrow E + E \quad (1)$$

$$E \rightarrow E * E \quad (2)$$

$$E \rightarrow (E) \quad (3)$$

$$E \rightarrow i \quad (4)$$

请给出字符串 $i + i * i$ 的推导过程。

两种语法树

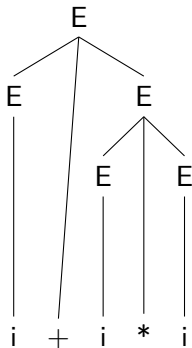


图 3: 推导 1 的语法树

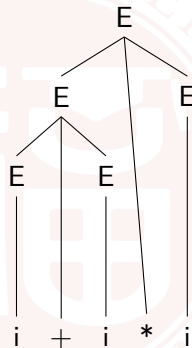


图 4: 推导 2 的语法树

最左与最右

- 最左推导 (Leftmost derivation): 在每一步推导中, 总是选择**最左边**的非终结符号进行替换。



最左与最右

- 最左推导 (Leftmost derivation): 在每一步推导中, 总是选择**最左边**的非终结符号进行替换。
- 最右推导 (Rightmost derivation): 在每一步推导中, 总是选择**最右边**的非终结符号进行替换。

最左与最右

- 最左推导 (Leftmost derivation): 在每一步推导中, 总是选择**最左边**的非终结符号进行替换。
- 最右推导 (Rightmost derivation): 在每一步推导中, 总是选择**最右边**的非终结符号进行替换。
- 最左推导的逆过程是最右归约——每次归约最右边的可归约符号串。

最左与最右

- 最左推导 (Leftmost derivation): 在每一步推导中, 总是选择**最左边**的非终结符号进行替换。
- 最右推导 (Rightmost derivation): 在每一步推导中, 总是选择**最右边**的非终结符号进行替换。
- 最左推导的逆过程是最右归约——每次归约最右边的可归约符号串。
- 最右推导的逆过程是最左归约——每次归约最左边的可归约符号串。

最左与最右

- 最左推导 (Leftmost derivation): 在每一步推导中, 总是选择**最左边**的非终结符号进行替换。
- 最右推导 (Rightmost derivation): 在每一步推导中, 总是选择**最右边**的非终结符号进行替换。
- 最左推导的逆过程是最右归约——每次归约最右边的可归约符号串。
- 最右推导的逆过程是最左归约——每次归约最左边的可归约符号串。
- 可归约符号串: 在归约过程中, 当前字符串中存在一个子串 α , 满足存在产生式 $A \rightarrow \alpha$, 则称 α 是可归约的。

文法的二义性

- 文法 $G[E]$ 是二义性文法 (ambiguous grammar), 因为字符串 $i + i * i$ 有两种不同的语法树, 从而有两种“不同的”推导过程¹。

¹不同的最左推导或最右推导

文法的二义性

- 文法 $G[E]$ 是二义性文法 (ambiguous grammar)，因为字符串 $i + i * i$ 有两种不同的语法树，从而有两种“不同的”推导过程¹。
- 二义性文法会导致编译器无法确定表达式的计算顺序，从而影响程序的正确执行。

¹不同的最左推导或最右推导

文法的二义性

- 文法 $G[E]$ 是二义性文法 (ambiguous grammar), 因为字符串 $i + i * i$ 有两种不同的语法树, 从而有两种“不同的”推导过程¹。
- 二义性文法会导致编译器无法确定表达式的计算顺序, 从而影响程序的正确执行。
- 解决方法: 引入运算符优先级和结合性规则, 或者重写文法以消除二义性。

¹不同的最左推导或最右推导

文法的二义性

- 文法 $G[E]$ 是二义性文法 (ambiguous grammar), 因为字符串 $i + i * i$ 有两种不同的语法树, 从而有两种“不同的”推导过程¹。
- 二义性文法会导致编译器无法确定表达式的计算顺序, 从而影响程序的正确执行。
- 解决方法: 引入运算符优先级和结合性规则, 或者重写文法以消除二义性。
- 可以通过以下文法消除二义性:

$$E \rightarrow E + T \mid T \quad (5)$$

$$T \rightarrow T * F \mid F \quad (6)$$

$$F \rightarrow (E) \mid i \quad (7)$$

¹不同的最左推导或最右推导

文法改写练习

练习

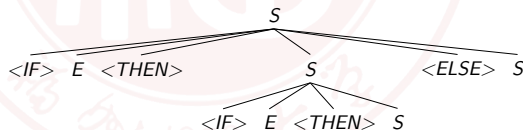
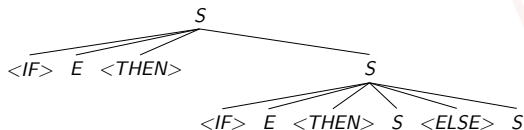
试改写下列文法以消除二义性：

$S ::= \langle IF \rangle E \langle THEN \rangle S \langle ELSE \rangle S$

$S ::= \langle IF \rangle E \langle THEN \rangle S$

$S ::= \langle OTHER \rangle$

...



二义性判定练习 1

练习

文法 G :

$$S \rightarrow aS \mid Sb \mid ab$$

问：请判定该文法是否为二义性文法。

二义性判定练习 2

例

文法 G :

$$S \rightarrow A \mid B$$

$$A \rightarrow aAb \mid \varepsilon$$

$$B \rightarrow aBb \mid ab$$

问：请判定该文法是否为二义性文法。

二义性判定练习 3

例

文法 G :

$$S \rightarrow aSb \mid abS \mid \varepsilon$$

问：请判定该文法是否为二义性文法，并给出证明。

二义性判定练习 4

例

文法 G :

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow (E) \mid i$$

问题：请证明该文法是无二义性的。

二义性判定练习 4（解）

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导

二义性判定练习 4 (解)

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导

二义性判定练习 4（解）

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导
- 归纳假设：假设所有长度小于 n 的字符串有唯一最左推导

二义性判定练习 4 (解)

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导
- 归纳假设：假设所有长度小于 n 的字符串有唯一最左推导
- 归纳步骤：考虑长度为 n 的字符串

二义性判定练习 4 (解)

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导
- 归纳假设：假设所有长度小于 n 的字符串有唯一最左推导
- 归纳步骤：考虑长度为 n 的字符串
 - 如果以 i 结尾，则 $F \Rightarrow i$ ，唯一

二义性判定练习 4 (解)

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导
- 归纳假设：假设所有长度小于 n 的字符串有唯一最左推导
- 归纳步骤：考虑长度为 n 的字符串
 - 如果以 i 结尾，则 $F \Rightarrow i$ ，唯一
 - 如果以 (E) 结尾，则 $F \Rightarrow (E)$ ，唯一

二义性判定练习 4 (解)

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导
- 归纳假设：假设所有长度小于 n 的字符串有唯一最左推导
- 归纳步骤：考虑长度为 n 的字符串
 - 如果以 i 结尾，则 $F \Rightarrow i$ ，唯一
 - 如果以 (E) 结尾，则 $F \Rightarrow (E)$ ，唯一
 - 如果以 $E + T$ 或 $T * F$ 结尾，则根据运算符优先级，唯一确定

二义性判定练习 4 (解)

解

- 证明思路：通过归纳法证明每个字符串有唯一的最左推导
- 基础情况：对于长度为 1 的字符串（如 i ），有唯一推导
- 归纳假设：假设所有长度小于 n 的字符串有唯一最左推导
- 归纳步骤：考虑长度为 n 的字符串
 - 如果以 i 结尾，则 $F \Rightarrow i$ ，唯一
 - 如果以 (E) 结尾，则 $F \Rightarrow (E)$ ，唯一
 - 如果以 $E + T$ 或 $T * F$ 结尾，则根据运算符优先级，唯一确定
- 结论：该文法是无二义性的

目录

- ① 语言
- ② 文法
- ③ 总结与作业



课堂小结

- 语言是传递信息的工具



课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介



课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法



课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次



课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次
- 推导和归约互为逆过程

课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次
- 推导和归约互为逆过程
- 语法树是表示推导过程的树形结构

课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次
- 推导和归约互为逆过程
- 语法树是表示推导过程的树形结构
- 推导过程中的非终结符替换顺序会不会影响语法树的形态

课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次
- 推导和归约互为逆过程
- 语法树是表示推导过程的树形结构
- 推导过程中的非终结符替换顺序会不会影响语法树的形态
- 无二义性文法的最左推导和最右推导唯一

课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次
- 推导和归约互为逆过程
- 语法树是表示推导过程的树形结构
- 推导过程中的非终结符替换顺序会不会影响语法树的形态
- 无二义性文法的最左推导和最右推导唯一
- 二义性文法会导致歧义，实用的文法应当不存在二义性

课堂小结

- 语言是传递信息的工具
- 程序设计语言是人与计算机之间的中介
- 文法是描述程序设计语言语法结构的形式化方法
- 文法由非终结符、终结符、产生式和起始符号组成，有四个不同层次
- 推导和归约互为逆过程
- 语法树是表示推导过程的树形结构
- 推导过程中的非终结符替换顺序会不会影响语法树的形态
- 无二义性文法的最左推导和最右推导唯一
- 二义性文法会导致歧义，实用的文法应当不存在二义性
- **二义性可以通过改写消除**

作业题

① 试对下如下语言构造相应的文法，并构造一个例子进行推导验证：

① $\{a(b^n)a \mid n = 0, 1, 2, \dots\}$

② $\{(a^n)(b^n) \mid n = 1, 2, 3, \dots\}$

其中，左右括号为终结符。

② 设文法 G 为：

$$N ::= D \mid ND$$

$$D ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

请给出句子 0125 和 783 的最左推导和最右推导。

用福州大学作业纸手写完成，下一次课前交。